Dijkstra's Algorithm

核心定义 | Core Definition

- **BFS** 寻找的是“段数最少”的路径（Shortest path in terms of segments）。
- **Dijkstra** 寻找的是“总权重最小”的路径（Shortest path in terms of total weight/cost）。

> **关键概念：权重 (Weights)**
>
> 边上的数字被称为权重。带权重的图称为**加权图 (Weighted Graph)**，不带权重的图称为**非加权图 (Unweighted Graph)**。

---

2. 算法的四个步骤 | The Four Steps of the Algorithm

要实现狄克斯特拉算法，你需要维护三张表：**图本身 (Graph)**、**开销表 (Costs)** 和 **父节点表 (Parents)**。

1. **找出“最便宜”的节点**：在尚未处理的节点中，找一个从起点出发开销最小的节点。
   - **Find the "cheapest" node**: Look for the node you can get to in the least amount of time/cost among unprocessed nodes.
2. **更新邻居的开销**：检查该节点的邻居，看看是否有前往它们的更短路径。如果有，更新其开销。
   - **Update the costs of neighbors**: Check if there's a cheaper path to the neighbors of this node. If so, update their costs.
3. **重复**：对图中的每个节点都这样做。
   - **Repeat**: Do this for every node in the graph.
4. **计算最终路径**：通过父节点表回溯，找到最短路径。
   - **Calculate the final path**: Backtrack using the parents table to find the path.

---

3. 数据结构建模 | Data Structure Modeling

在 Python 中，我们通常使用**嵌套字典 (Nested Dictionaries)** 来存储加权图。

```python
# 建模图 (Modeling the Graph)
graph = {}
graph["start"] = {"a": 6, "b": 2}
graph["a"] = {"fin": 1}
```

```
graph["b"] = {"a": 3, "fin": 5}
graph["fin"] = {} # 终点没有邻居
```

- **开销表 (Costs Table)**：存储从起点到该节点的已知最低开销。初始时，不知道开销的节点设为 $infinity$ (无穷大)。
- **父节点表 (Parents Table)**：记录路径。例如 `parents["a"] = "start"` 表示要到达 A，上一站是起点。

---

4. 局限性：负权重 | Limitation: Negative Weights

这是 Dijkstra 的“阿喀琉斯之踵”。

- 如果图中包含**负权边 (Negative-weight edges)**，狄克斯特拉算法就会失效。
- **原因**：该算法假设一旦处理了某个节点，就没有前往该节点的更短路径。但负权边会打破这个假设。
- **解决方案**：使用 **贝尔曼-福德算法 (Bellman-Ford algorithm)**。

> **Dijkstra's logic**: "I've found the cheapest way to Node A, so I'm done with it."
>
> **Negative weight's logic**: "Wait! If you go through Node C, I'll actually give you money back, making Node A even cheaper!"

---

5. 复杂度分析 | Complexity Analysis

对于未来的 Honors 学生，你需要知道：

- 使用普通的列表寻找最小开销节点，复杂度约为 $O(V^2)$。
- 使用**优先队列/小顶堆 (Priority Queue / Min-heap)** 优化后，复杂度可以达到 $O(E \log V)$。
  - V = 节点数 (Vertices)
  - E = 边数 (Edges)

💡 总结 | Summary

- **BFS** 用于非加权图，找步数最少。
- **Dijkstra** 用于加权图，找开销最少。

- **Bellman-Ford** 用于包含负权边的图。